

Building a Reproducible Scholarly Search Toolkit

Episode 9: Exponential Backoff & SQLite Caching

Nexus Scholar Suite

AcademicHttpClient Architecture



HTTP Client Implementation

```
class AcademicHttpClient:
    DEFAULT_TIMEOUT: float = 30.0

    def __init__(self, name: str, rate_limit: float, cache_enabled: bool = True):
        self.name = name
        self.rate_limiter = RateLimiter(rate_limit)

        # 1. SQLite Caching Session
        if cache_enabled:
            self.session = requests_cache.CachedSession(
                backend='sqlite', expire_after=timedelta(days=30)
            )

        # 2. Exponential Retry Strategy
        retry = Retry(total=4, backoff_factor=2.0,
                      status_forcelist=[429, 500, 502, 503, 504])
        self.session.mount("https://", HTTPAdapter(max_retries=retry))
```